1ª AVALIAÇÃO · PROGRAMAÇÃO PARALELA E DISTRIBUÍDA

Análise Paralela de Transações Financeiras

Como dividir 10 milhões de transações entre várias threads em Java, com ExecutorService e afinidade de processador

Helena Vieira · Camila Rocha Ribeiro · Maria Clara Pastor
IFSP Câmpus Jacareí · JCRPGPD · 4º semestre de 2026

CONTRIBUIÇÃO INDIVIDUAL

Quem fez o quê na equipe?

Helena Vieira

Dados e agregação

- Modelo da transação
- Gerador com 4 distribuições
- Soma, média, maior e menor valor
- Sequencial, Executor e Affinity

Camila Ribeiro

Suspeitas e categorias

- Regras de fraude e filtros
- Estatísticas por categoria
- Junção dos resultados em paralelo
- Versões sequencial e concorrente

Maria Clara Pastor

Benchmark e afinidade

- Infraestrutura de concorrência
- Afinidade de CPU via OpenHFT
- Coleta de métricas e Main

INTRODUÇÃO

Por que analisar transações em paralelo?

Cenário

Pix, cartões e transferências geram dezenas a centenas de milhões de registros por dia.

Problema

Processar de forma sequencial leva a tempos proibitivos e dificulta detectar fraudes e consolidar contas.

Proposta

Computação paralela em Java com ExecutorService e afinidade de CPU via OpenHFT Affinity.

INTRODUÇÃO

Por que dá para dividir esse trabalho?

Paralelismo de dados: a mesma tarefa é aplicada a pedaços diferentes dos dados, ao mesmo tempo.

1 **Transações independentes**
A transação i não depende do resultado da transação i − 1.

2 **Mesma lógica em subconjuntos disjuntos**
Somar, checar suspeitas e agrupar categorias, aplicados a fatias diferentes do vetor.

3 **Sincronização só no final**
Sem dependência entre fatias, só a etapa de redução precisa combinar resultados.

OBJETIVO E QUESTÃO CENTRAL

Qual é o impacto do ExecutorService e da afinidade de processador na análise de grandes volumes de transações?

Para responder, comparamos três jeitos de rodar o mesmo programa:

1 Sequencial

Uma única thread faz todo o trabalho. É a nossa referência de tempo.

2 ExecutorService

Várias threads dividem as transações e trabalham ao mesmo tempo.

3 Executor + Affinity

Igual ao anterior, mas cada thread fica presa a um núcleo do processador.

DADOS SINTÉTICOS

Transação modelada com 4 distribuições

`id · cliente · data · valor · categoria · cidade · estado · tipo`

Uniforme

Cidade, tipo de pagamento, data e compras comuns (R$ 10 a R$ 500).

Normal

Alimentação e Transporte, em torno de R$ 150 (desvio de R$ 40).

Exponencial

Saúde e Educação: muitos valores pequenos e poucos grandes.

Pareto (cauda longa)

Eletrônicos, até R$ 100.000: simula compras raras e suspeitas.

PROBLEMA COMPUTACIONAL

O que o programa calcula?

Nº	Análise	O que calcula
1	**Soma total**	A soma de todos os valores
2	**Ticket médio**	O valor médio por transação
3	**Suspeitas**	Valor acima de R$ 10.000, ou acima de R$ 5.000 em Eletrônicos
4	**Por categoria**	Quantidade e valor total em cada uma das 5 categorias
5	**Extremos**	O maior e o menor valor

As 5 categorias: Alimentação, Transporte, Eletrônicos, Saúde e Educação.

EXEMPLO COM 4 THREADS

Como dividimos as transações entre as threads?

Tamanho de cada bloco = total / nº de threads → **10.000.000 ÷ 4 = 2.500.000**

Thread 0
0 até 2.499.999

Thread 1
2.500.000 até 4.999.999

Thread 2
5.000.000 até 7.499.999

Thread 3
7.500.000 até 9.999.999

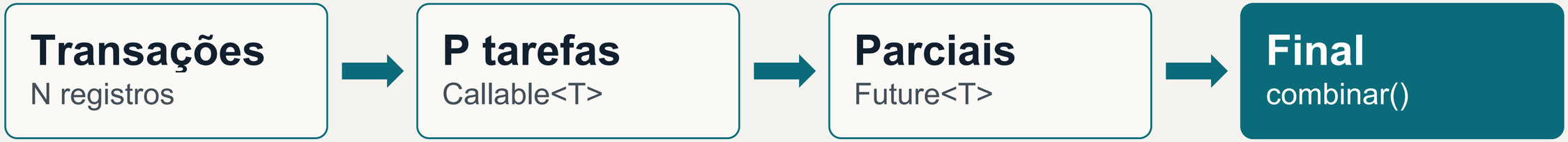
Mesmo trabalho, fatias diferentes
Todas as threads rodam o mesmo código, cada uma no seu bloco.

Por que blocos vizinhos?
Dados em sequência na memória são lidos mais rápido (cache).

E se a conta não for exata?
A última thread fica com o que sobrar.

EXECUTORSERVICE

Como as threads trabalham e juntam os resultados?



Pool fixo: newFixedThreadPool(P) cria uma equipe fixa de P threads, reaproveitada em vez de criar uma thread por tarefa.

Future: Funciona como um recibo: a thread principal usa get() para buscar o resultado de cada tarefa quando ela termina.

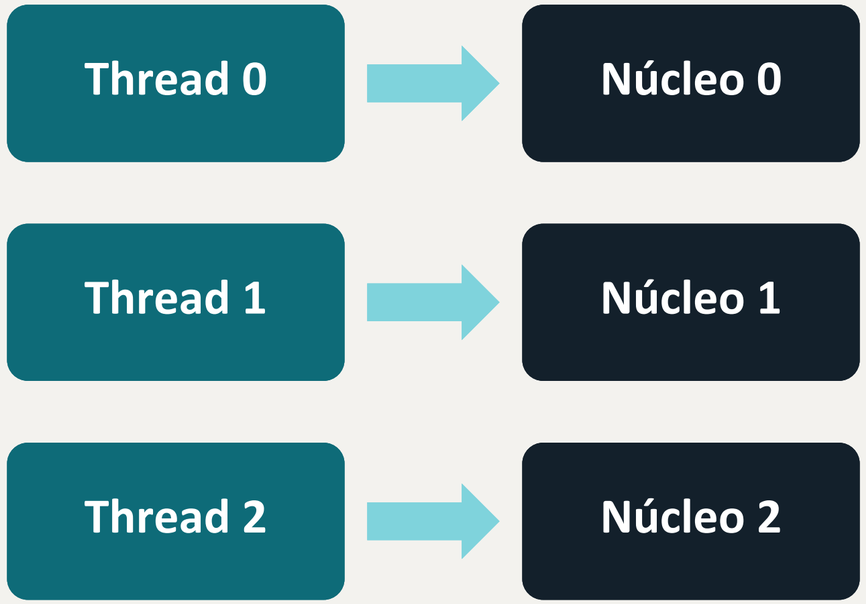
combinar(): Soma os resultados parciais, um por vez. Cada thread tem o seu, então nenhuma atrapalha a outra.

```
for (Future f : futuros)
    resultadoFinal.combinar(f.get());
```

O que é fixar cada thread em um núcleo?

Normalmente, o Windows decide em qual núcleo cada thread roda e pode trocá-la de núcleo no meio do trabalho.

Com afinidade, cada thread fica presa ao seu núcleo e não perde os dados que já guardou no cache.



OpenHFT
`Affinity.setAffinity(cpuId)`
cpuId = nº da thread mod nº de CPUs

METODOLOGIA DOS EXPERIMENTOS

Como medimos o desempenho?

8
processadores lógicos (Intel Core)

1 e 10
milhões de transações

1 a 8
threads (1, 2, 4 e 8)

3
configurações comparadas

Dois tamanhos: Para ver se o paralelismo compensa com pouco e com muito dado.

Ambiente: Windows 64-bit, memória DDR4, Java 17 (JDK HotSpot).

RESULTADOS · 10 MILHÕES DE TRANSAÇÕES

Quanto tempo cada configuração levou?

Configuração	Threads	Tempo (ms)	Speedup	Eficiência
Sequencial	1	62	1,00	100,0%
Executor	2	78	0,79	39,7%
Executor	4	36	1,72	43,1%
Executor	**8**	**29**	**2,14**	**26,7%**
Executor + Affinity	2	65	0,95	47,7%
Executor + Affinity	4	41	1,51	37,8%
Executor + Affinity	8	33	1,88	23,5%

Speedup

tempo sequencial ÷ tempo paralelo

Quantas vezes ficou mais rápido. Abaixo de 1 = mais lento que o sequencial.

Eficiência

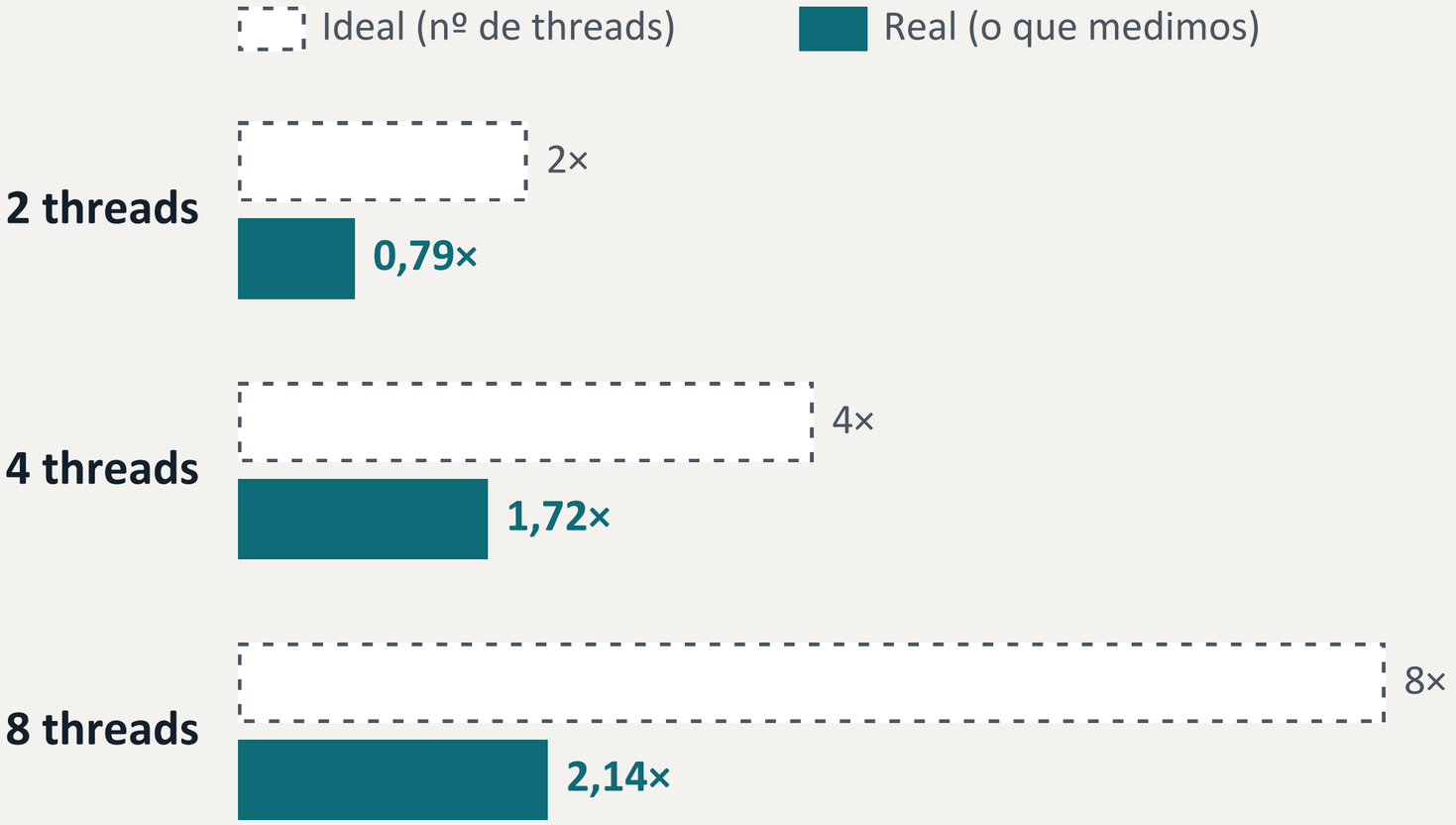
speedup ÷ nº de threads

Quanto cada thread rendeu. 100% = aproveitamento perfeito.

Destaque: melhor resultado com 8 threads, de 62 para 29 ms (2,14×).

SPEEDUP · 10 MILHÕES DE TRANSAÇÕES

O ganho cresce, mas fica longe do ideal



2,14×

maior speedup (8 threads)

8×

seria o ideal com 8 threads

ANÁLISE DOS RESULTADOS

Por que não ficou 8× mais rápido?

Com 8 threads ficou **2,14×** mais rápido. O ideal seria **8×**.

1. Custo de organizar

Overhead

Criar as tarefas, distribuir os blocos e esperar cada uma terminar leva tempo.

2. Parte que não se divide

Lei de Amdahl

Juntar os resultados no combinar() é feito só pela thread principal.

3. Memória compartilhada

Gargalo de memória

As contas são simples: as threads passam mais tempo buscando dados na mesma RAM.

Por isso a eficiência cai: **43%** com 4 threads e **27%** com 8, pois no Hyper-Threading dois processadores lógicos dividem um núcleo.

TAMANHO DO CONJUNTO DE DADOS

Com 1 milhão, o paralelismo não compensou

Configuração	Threads	Tempo (ms)	Speedup
Sequencial	1	6	1,00
Executor	2	8	0,75
Executor	4	6	1,00
Executor	8	7	0,86
Executor + Affinity	2	6	1,00
Executor + Affinity	4	7	0,86
Executor + Affinity	8	30	0,20

1 milhão: 6 ms

Nenhuma versão paralela foi mais rápida que o sequencial.

10 milhões: 62 → 29 ms

Aí sim o paralelismo valeu: 2,14× mais rápido.

Com pouco dado, o custo de organizar as threads é maior do que o ganho de dividir.

IMPACTO DA AFINIDADE · 10 MILHÕES

A afinidade ajudou só com 2 threads

Threads	Sem afinidade	Com afinidade	Diferença
2	78 ms	65 ms	**13 ms mais rápido**
4	36 ms	41 ms	**5 ms mais lento**
8	29 ms	33 ms	**4 ms mais lento**

Com 2 threads, ajudou

Cada thread manteve seus dados no cache: o speedup subiu de 0,79 para 0,95.

Com 4 e 8, atrapalhou

O Windows já distribui bem as threads, e as fixas no núcleo 0 sofrem interferência do sistema.

REQUISITO IMPORTANTE

Por que não criar uma thread para cada transação?

10 milhões de threads

Falta memória: cada thread reserva cerca de 1 MB, ou seja, uns 10 TB no total. O Java para com OutOfMemoryError.
Fila de espera: só 8 threads rodam de verdade ao mesmo tempo. O resto fica esperando, e o processador perde tempo trocando de uma para outra.

Granularidade grossa

Uma tarefa por thread (no máximo 8.)
Cada uma pega um bloco grande do vetor.
Quase sem custo: 8 threads usam só uns 8 MB.

CONCLUSÃO

O que aprendemos?

62 → 29 ms

Com ExecutorService e 8 threads: 2,14× mais rápido com 10 milhões de transações.

78 → 65 ms

A afinidade ajudou com 2 threads; com 4 e 8, o ganho não apareceu.

Equilíbrio

O melhor resultado depende de quantas tarefas criar, do tamanho de cada uma e da máquina.

Dividir o trabalho entre threads vale a pena com muitos dados, mas cada thread a mais rende menos que a anterior.

Obrigada!

Helena Vieira · Camila Rocha Ribeiro · Maria Clara Pastor